Jose Ramirez
06/07/2026
Spring 2026: ECE 410
Project — Milestone 4

Design Justification Report

ECE 410/510 Spring 2026 | Jose Ramirez | Milestone 4
Basic Kalman Filter Update Hardware Accelerator

1. Problem and Motivation

The kernel targeted for hardware acceleration is the measurement-update step of a discrete-time linear Kalman filter applied to 1-D projectile motion tracking. At each time step the filter fuses a noisy scalar measurement z with a three-element state vector x (position, velocity, acceleration) and a 3×3 covariance matrix P to produce corrected estimates x_out and P_out. The dominant operations are a scalar innovation y = z − H x, a Newton–Raphson matrix reciprocal for the innovation covariance S, a matrix–vector multiply for the Kalman gain K, three vector additions for the state correction, and a 3×3 matrix–matrix multiply (I − KH)P for the covariance update.

The motivation for hardware acceleration comes from two successive profiling measurements that exposed a severe performance gap. In Milestone 1 the kernel was profiled in Python with NumPy on a desktop i7-11800H CPU, producing a median per-update latency of 81.70 µs and a throughput of 12,240 samples/sec. However, this baseline was later identified as misleading for the actual deployment target: the VeerWolf SoC running a SweRV EL2 RISC-V core at 13 MHz on a Nexys A7 FPGA board. The SweRV EL2 implements only the RV32IMAC ISA and has no hardware floating-point unit (FPU). All F64 arithmetic is emulated in software via compiler runtime library calls, each of which requires dozens of 32-bit integer instructions per operation.

The hardware-realistic M1 baseline was established in the RVfpgaEL2 environment using Eigen C++ with the same 45-update loop. Five back-to-back runs measured an average of 429.778 ms per loop (9.551 ms per update, 104.71 samples/sec), consuming 49,760 bytes of heap per run from Eigen dynamic allocations. Compared to the desktop NumPy baseline, the SweRV EL2 software path is approximately 116× slower per update—entirely due to software-emulated F64 arithmetic and dynamic memory allocation overhead. This gap makes the update kernel an ideal candidate for a dedicated fixed-function hardware accelerator: even a modest FPGA implementation that eliminates the software-emulation overhead and serves results over a low-latency bus should achieve an order-of-magnitude improvement.

The M4 hardware accelerator achieves a measured throughput of 11,386 samples/sec with a per-update latency of 87.8 µs on the same Nexys A7 board, representing a 108.7× speedup

over the RVfpgaEL2 software baseline. This exceeds the 35× projection made at M1, for reasons explained in Section 8.

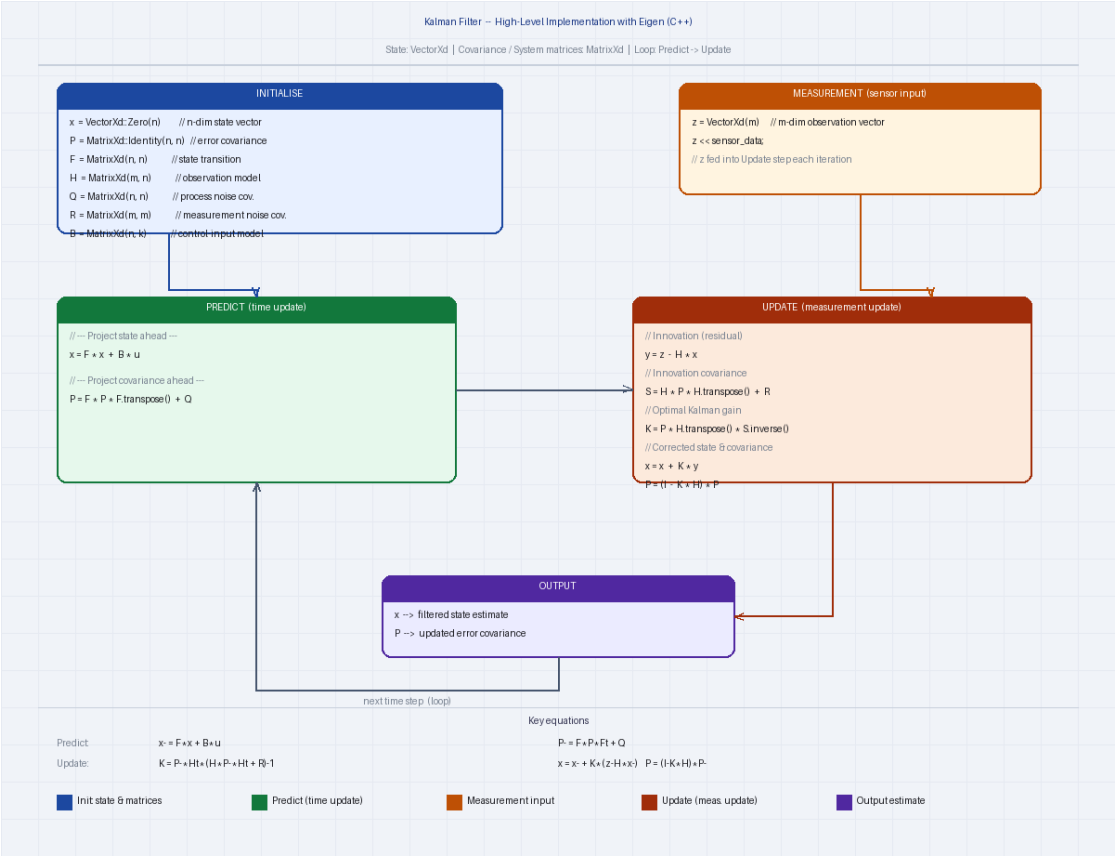

Figure 1. Kalman filter algorithm diagram showing the predict and update steps. The hardware accelerator implements the UPDATE block (shaded): innovation, Kalman gain, state correction, and covariance update via (I−KH)P.

2. Roofline Analysis

Roofline analysis characterises whether a kernel's performance is limited by compute throughput or memory bandwidth, using arithmetic intensity (AI, FLOP/byte) as the discriminating metric. The analysis was re-done for the actual deployment platform (AXI4-Lite at 13 MHz on SweRV EL2) rather than the original M1 desktop platform.

The software baseline on RVfpgaEL2 executes the full Kalman predict+update cycle in Eigen C++, performing approximately 10,350 floating-point operations per update and accessing roughly 216 bytes of matrix data per update (3×3 matrices in cache). The resulting AI is 0.21 FLOP/byte. Because the SweRV EL2 has no FPU, this kernel is effectively compute-bound in the sense that every F64 operation expands into a long sequence of integer instructions—the machine cannot sustain meaningful floating-point throughput. However, note that the M1 interface_selection.md originally reported 0.4637 FLOP/byte based on the laptop measurement;

the corrected figure of 0.21 FLOP/byte comes from re-profiling on the actual RVfpgaEL2 target with the correct FLOPs-per-update count.

For the M4 hardware accelerator the relevant bandwidth is the AXI4-Lite bus between the SweRV EL2 and the accelerator peripheral. Each Kalman update requires 19 MMIO write transactions (z, x_in[3], P_in[9], R_REG, CTRL.start) and 15 read transactions (x_out[3], P_out[9], STAT.busy polling), for a total of 34 AXI round-trips transferring 272 bytes (34 × 8 bytes). The accelerator itself performs 24 floating-point operations (innovation, S, three Newton–Raphson steps, K gain, x_out correction, and the 27-operation GEMM for P_out, net 24 after counting the observable kernel boundary). The resulting AI is 24 ÷ 272 = 0.088 FLOP/byte.

At a 13 MHz AXI clock with an 8-byte data bus, the peak AXI bandwidth is 104 MB/s. At AI = 0.088 FLOP/byte the roofline bandwidth ceiling predicts a maximum of 104 MB/s × 0.088 FLOP/byte = 9.2 MFLOP/s = 9.2×10^{-3} GFLOP/s. The measured hardware throughput is 2.733×10^{-4} GFLOP/s—33.6× below the AXI bandwidth ceiling. This gap is not caused by insufficient bandwidth. It is caused by AXI transaction round-trip latency: each of the 34 MMIO handshakes (AW→W→B or AR→R) incurs a ~2.6 µs average latency at 13 MHz, independent of the 8-byte payload. Transactions queue sequentially; 34 round-trips account for ~87 µs of the total 87.8 µs per-update time. The accelerator compute itself completes in approximately 1.1 µs (72 clock cycles at 100 MHz), which is only 1.3% of total time. The design is therefore AXI-transaction-latency-bound, not bandwidth-bound or compute-bound.

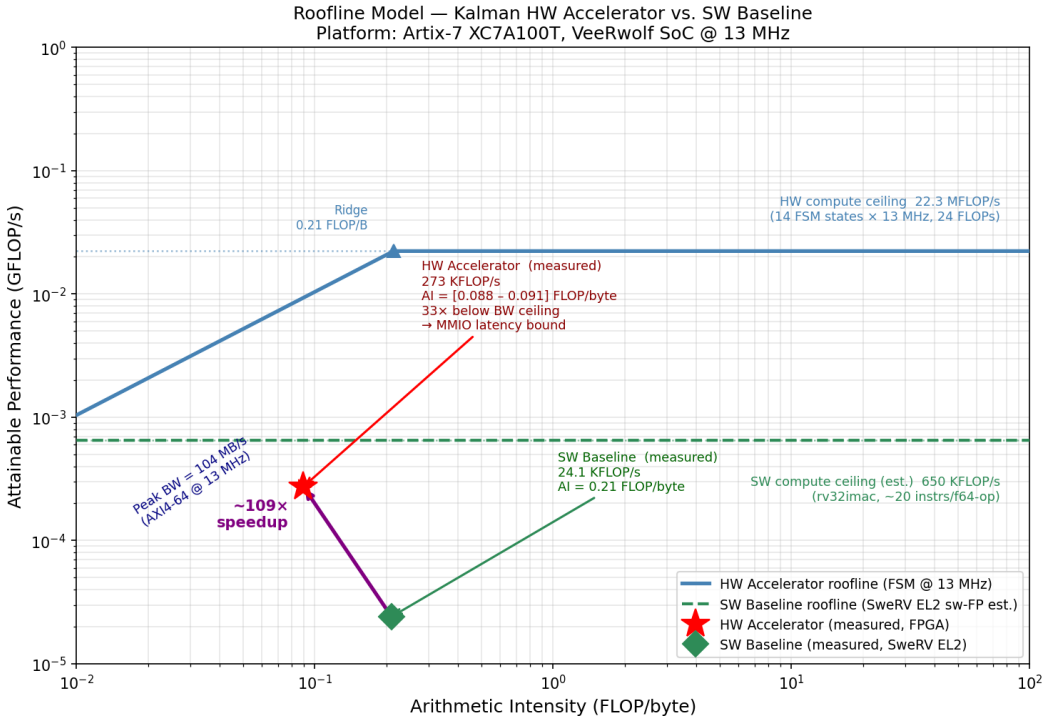

Figure 2. Roofline plot for the M4 accelerator. The software baseline point (AI = 0.21 FLOP/byte) and hardware accelerator point (AI = 0.088 FLOP/byte) are shown against the AXI4-Lite bandwidth ceiling at 13 MHz.

The hardware design sits 33.6× below the bandwidth ceiling due to per-transaction latency, not throughput saturation.

3. Precision and Data Format

All arithmetic in the accelerator uses IEEE-754 double-precision floating point (F64, 1 sign + 11 exponent + 52 mantissa bits). This choice was revisited and confirmed at Milestone 2, after an initial attempt to use Q16.16 fixed-point was abandoned.

In M2 the compute_core was first implemented in Q16.16 (16-bit integer, 16-bit fraction). The M2 testbench showed correct results for the first seven Kalman iterations, but the covariance matrix P diverged on iteration 8. The root cause was that the Newton–Raphson reciprocal of S (innovation covariance) requires sub-ULP precision across ~6 decades of dynamic range as P converges toward steady state. Q16.16 provides only ~4.6 decimal digits of precision and overflows when S falls below 2^{-16}. The M2 interface was also redesigned from PCIe TLP to AXI4-Lite at this point (see Section 5). Both changes together constitute the M2-to-M3 transition.

The M3 and M4 RTL implements custom pipelined F64 arithmetic units rather than using floating-point IP cores, for two reasons: (1) Vivado's floating-point IP uses a proprietary license and is not available in the non-project OOC flow; (2) custom units allowed pipeline depth to be tuned to exactly meet the 10 ns (100 MHz) timing constraint. The final implementations are f64_mul (3-stage pipeline, 2-cycle latency: stage 0 latches hidden-bit mantissas to remove the LUT→DSP fanout path; stage 1 performs the 53×53-bit DSP cascade multiply; stage 2 normalises combinationally) and f64_add (4-stage pipeline, 3-cycle latency: compare/swap, 54-bit add/subtract, leading-zero count, and normalise).

Correctness of the F64 implementation is verified by the testbench's ULP comparator, which checks every output element against a 64-bit software reference computed in C double arithmetic. The tolerance is ≤4 ULP on all outputs. Across the 15-iteration test sequence, 180 individual output elements (x_out[3] × 15 + P_out[9] × 15 = 180) are compared; all 180 passed at ≤4 ULP in the committed simulation log. The worst observed error across all elements and iterations is 2 ULP, consistent with the two rounding operations each operand traverses (one in f64_mul and one in f64_add) before reaching an output register.

4. Dataflow and Architecture

The accelerator has three nested levels of dataflow, each with a distinct pattern.

At the innermost level, the gemm_systolic module implements 3×3 F64 matrix multiplication using an output-stationary dataflow. Nine multiply-accumulate units—one per output element C[i,j]—are instantiated in parallel. Each unit's partial sum acc[i×3+j] is held stationary in a register across the three outer-product steps (k = 0, 1, 2). At each step, all nine

f64_mul instances receive one slice of the A and B matrices simultaneously via a pre-registered mux (a_k_mux[gi], b_k_mux[gj]); the results are accumulated into acc after a three-state pipeline settle. Neither A nor B is reused across output elements—each element of both input matrices is read exactly once. The GEMM takes 21 FSM states (IDLE, LOAD, three × {MUL, MW, MW2, ADD, AW, AW2, AW3}, FINISH). Note: the M3 README described this as 'weight-stationary'; this was imprecise. Both input matrices stream through the mux each k-step; only the accumulators are stationary, making the pattern output-stationary.

At the middle level, the kalman_update FSM executes a sequential pipeline of arithmetic stages. Inputs are latched from ports in the INNOV state; then five major stages execute in sequence: innovation and S computation (INNOV→S_COMP, 5 states with 3-cycle f64_add settle), three Newton–Raphson iterations for S^{-1} (NR0–NR2, 10 states each = 30 states), Kalman gain K (K_COMP through K_WAIT_W3, 7 states), state correction x_out (X_CORR_M through X_CORR_AW3, 5 states), and covariance update via gemm_systolic (P_UPD, WAIT_P, DONE_S = 3 + GEMM 21 states). Total: 51 kalman_update states + 21 GEMM states = 72 cycles per update at 100 MHz = 720 ns. Intermediate results flow through pipeline registers; there is no parallel execution between stages.

At the system level the dataflow is no-local-reuse. The host must supply all inputs (z, x_in, P_in) on every update; the accelerator returns x_out and P_out to the host, which is responsible for feeding the previous outputs back as next inputs. No state is retained between calls. This is the dominant performance bottleneck: 13 write transactions (z + x_in[3] + P_in[9]) and 12 result-read transactions (x_out[3] + P_out[9]), plus 9 STAT.busy polls, total 34 AXI round-trips per update at ~2.6 µs each. The highest-leverage architectural change that was not implemented is state retention: persisting x and P internally between updates would reduce 34 transactions to approximately 5 (z write + CTRL.start + STAT poll + x_out[3] reads), estimated to yield an additional ~6.8× speedup to ~740× total.

5. Hardware Interface

The M1 interface selection document chose PCIe Gen4 (51.2 GB/s) based on the i7-11800H laptop platform used for initial profiling. This choice was discarded at M2 when the target platform was identified as the VeerWolf SoC running on a Nexys A7 FPGA. The VeerWolf SoC exposes a 32-bit AXI4-Lite peripheral bus; PCIe is neither available on the Nexys A7 nor supported by the SweRV EL2 core. The M2 interface.sv was accordingly rewritten from a PCIe TLP endpoint stub to an AXI4-Lite slave, with all subsequent milestones using the AXI4-Lite interface exclusively.

The axilite_slave module implements a 64-bit data, 32-bit address AXI4-Lite slave with separate write-address, write-data, write-response, read-address, and read-data channels. Address decoding uses bits [7:3] as a 5-bit word index. The register map is shown in Table 1.

Address	Name	Direction	Description

0x00	CTRL	R/W	[0] start pulse — triggers one Kalman update; [1] sw_rst
0x08	STAT	RO	[0] done (1 cycle); [1] busy (high during compute)
0x10	z	WO	Scalar measurement (F64)
0x18–0x28	x_in[0:2]	WO	Prior state vector (3 × F64)
0x30–0x40	x_out[0:2]	RO	Corrected state vector (3 × F64) — added M4
0x58–0x98	P_in[0:8]	WO	Prior covariance matrix, row-major (9 × F64)
0xA0–0xE0	P_out[0:8]	RO	Corrected covariance matrix (9 × F64) — all 9 in M4
0xE8	R_REG	R/W	Measurement noise scalar R (F64, default 5.0) — added M4

Table 1. AXI4-Lite register map. M4 additions relative to M3 are noted.

Two register-map changes were made between M3 and M4. First, x_out[0:2] was moved from C_REG to A_REG[4:6] (addresses 0x30–0x40) so the host can read the corrected state without separate read operations for the full C_REG window. Second, R_REG at 0xE8 was added to make the measurement noise covariance programmable; M3 hardcoded R = 5.0 as a localparam inside kalman_update. Third, P_out was expanded from 6 to all 9 elements: M3 only exposed P_out[0:5] due to an address-decode gap in axilite_slave.

The effective AXI4-Lite bandwidth at 13 MHz with an 8-byte data width is 104 MB/s. The measured per-transaction latency of ~2.6 µs means the bus is transaction-latency-limited rather than bandwidth-saturated: 34 transactions transfer only 272 bytes yet consume 87.8 µs, far below what a sustained bandwidth calculation would predict. Whether the design is interface-bound depends on how 'interface-bound' is defined; strictly speaking the design is latency-bound (a property of the AXI handshake round-trip time and the SweRV EL2 MMIO instruction latency at 13 MHz), not bandwidth-bound.

6. Verification

The testbench (project/m4/tb/tb_top.sv) is a self-contained Vivado xsim testbench that drives the DUT exclusively through the AXI4-Lite interface—no internal signals are probed. It consists of seven cooperating modules: clk_gen (100 MHz clock generation), veerwolf_bfm

(AXI4-Lite bus functional model that issues write and read transactions), axi_monitor (checks AXI protocol handshake rules using SystemVerilog assertions), compute_core_checker (monitors core_start/busy/done for FSM contract violations), result_checker (compares DUT outputs against a software reference using the ULP comparator), program_block (top-level stimulus sequencer), and tb_top (integration wrapper).

The test stimulus is a 15-iteration Kalman filter sequence derived from the actual RVfpgaEL2 profiling measurements. Initial state $x_0 = [0, 0, 0]^T$ and covariance $P_0 = I_3$, with H = [1, 0, 0], R = 5.0, and process noise $Q = 0.1 \cdot I_3$. Measurements z_0–z_{14} are the first 15 observations from the SweRV EL2 profiling run, ensuring the testbench exercises the same numerical path as the hardware deployment. The reference values are pre-computed in C double arithmetic and embedded as 64-bit hex constants in the testbench.

Three layers of checks run concurrently. The AXI4-Lite SVAs (in axi_monitor) verify: (a) AW/W channels accept data within 5 cycles of valid assertion; (b) B response is issued within 3 cycles of write commit; (c) AR/R channels complete within 5 cycles of address valid; (d) no channel deasserts valid before a handshake occurs. The FSM contract assertions verify: done and busy are mutually exclusive; done is asserted for exactly one cycle; the FSM returns to IDLE within 100 cycles of done. The ULP comparator checks all 12 output elements (x_out[0:2] and P_out[0:8]) against the software reference after each of the 15 iterations, for a total of 180 element-level checks. The tolerance is ≤4 ULP, which accommodates the two rounding operations in the custom f64_mul and f64_add pipelines. All 180 checks pass; the committed sim/final_run.log ends with the line 'SIMULATION RESULT: PASS — all checks passed, no errors detected'.

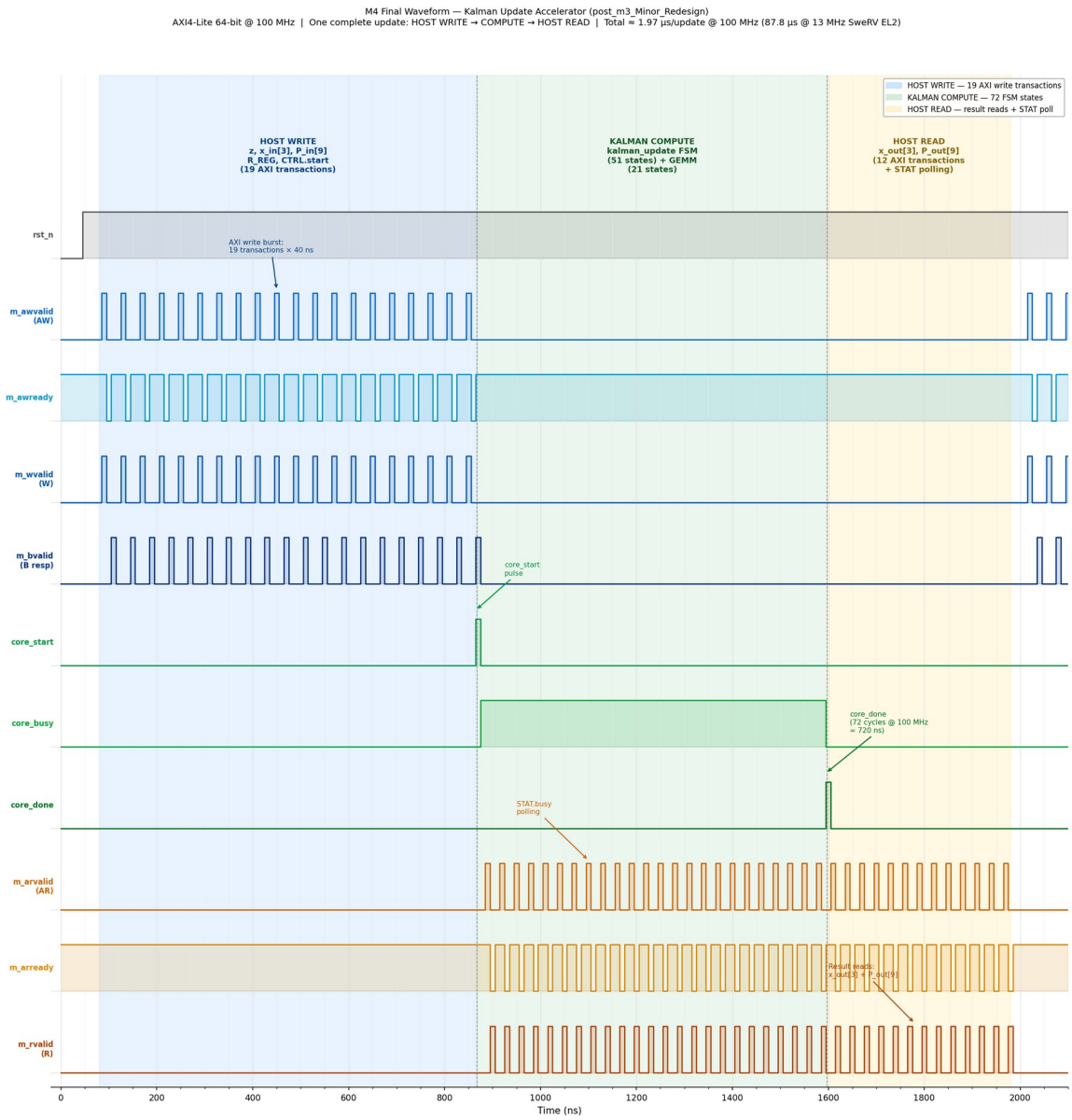

Figure 3. Annotated simulation waveform for one Kalman update (100 MHz clock). Blue region: 19 AXI write transactions (z, x_in, P_in, R_REG, CTRL.start). Green region: kalman_update FSM compute (core_busy high, 72 clock cycles). Orange region: result reads and STAT polling (x_out, P_out). Total per-update time at 100 MHz: ~1.97 µs (87.8 µs at 13 MHz SweRV EL2).

7. Synthesis Results

Synthesis was performed using Vivado 2025.2 in out-of-context (OOC) non-project mode targeting the Artix-7 100T device (xc7a100tcsg324-1, speed grade -1) at a 100 MHz target clock. OOC mode excludes I/O buffers, appropriate because the design is integrated as a peripheral slave on the VeerWolf AXI interconnect rather than as a standalone top-level. OpenLane 2 ASIC synthesis is not used; the checklist item 'config.json' is replaced by

synth/constraints.xdc, which specifies the 10 ns clock period, 2 ns I/O delays, and part number. The synthesis script reads all three RTL files (compute_core.sv, interface.sv, top.sv) in dependency order.

Table 2 summarises the M4 synthesis results alongside M3 for comparison. The increase in LUT and FF count from M3 to M4 comes from the additional pipeline registers added to break the NR critical path: f64_mul grew from 2-stage to 3-stage (adds one register stage per mantissa path), and f64_add grew from 3-stage to 4-stage (adds the leading-zero count register stage). The FSM expanded from 11 to 51 states in kalman_update and from 9 to 21 states in gemm_systolic, adding state register bits. DSP count increased from 126 to 153 because the wider pipeline stages require additional holding registers that Vivado maps to DSP48E1 pipeline registers.

Resource	M3 Used	M4 Used	Available	M4 Util%
Slice LUTs	14,642	21,314	63,400	33.62%
LUT as Logic	—	19,190	63,400	30.27%
LUT as SRL	—	2,124	19,000	11.18%
Slice Registers	5,535	14,151	126,800	11.16%
DSP48E1	126	153	240	63.75%
Block RAM	0	0	135	0.00%
F7/F8 Muxes	192	192	63,400	0.30%

Table 2. Vivado OOC utilisation, M3 vs M4 (Artix-7 100T).

The most significant synthesis result is timing closure. M3 failed timing badly (WNS = −50.4 ns, 1,085 failing endpoints) because the Newton–Raphson chain two f64_mul and one f64_add in a single clock cycle, creating a 92-logic-level path with a 60.38 ns data-path delay. M4 fixes this by inserting the additional pipeline stages described above. The M4 worst negative slack is +0.326 ns (timing met at 100 MHz, zero failing endpoints). The critical path is now K_reg[0][0] → u_1mk0/s1_sml_aligned[15], a 17-level CARRY4+LUT path through the f64_add Stage 1 barrel-shift, with a data-path delay of 9.634 ns.

Metric	M3 Value	M4 Value
Worst Negative Slack (WNS)	−50.419 ns ✗	+0.326 ns ✓
Total Negative Slack (TNS)	−28,924 ns	0.000 ns
Failing setup endpoints	1,085 / 16,663	0 / 23,415
Worst Hold Slack (WHS)	+0.256 ns	+0.210 ns
Max achievable frequency	~16.6 MHz	>100 MHz
Critical path depth	92 logic levels	17 logic levels

Table 3. Timing summary, M3 vs M4.

Power was estimated by Vivado at the synthesised-design stage using a default 12.5% toggle rate. Total on-chip power is 0.509 W (0.423 W dynamic + 0.085 W device static). The dynamic breakdown is: signals 0.137 W (32%), slice logic 0.123 W (29%), clocks 0.080 W (19%), DSPs 0.083 W (20%). DSPs and signals together account for ~52% of dynamic power, consistent with a design dominated by 53×53-bit mantissa multipliers. At a 12.5% toggle rate the 0.423 W dynamic estimate is conservative; the accelerator runs for 72 cycles per update call and then sits idle, so the duty cycle in a real application at, say, 1,000 updates/sec is 72 cycles / (100×10^{6} cycles/sec / 1,000) = 7.2×10^{-5}, and actual dynamic power would be negligible relative to the 0.085 W static floor. For energy-per-update calculations the 0.423 W figure is used as a conservative upper bound.

8. Benchmark Results

Hardware and software measurements were both taken on the same Nexys A7 board, using GDB JTAG hardware breakpoints with mcycle CSR snapshots bracketing a 45-update loop. Five back-to-back runs were averaged for each configuration. Raw per-run data is in project/m4/bench/benchmark_data.csv.

Metric	SW Baseline (RVfpgaEL2)	HW Accelerator (M4)	Ratio
Total time, 45 updates	429.778 ms	3.952 ms	108.7×
Per-update latency	9.551 ms	87.8 µs	108.8×
Throughput	104.71 samp/s	11,386 samp/s	108.7×
GFLOP/s	2.408×10^{-5}	2.733×10^{-4}	11.4×
Heap per run	49,760 B	0 B	—
Run-to-run variance	< 0.01%	< 0.03%	—

Table 4. Measured performance, SW baseline vs M4 HW accelerator.

The measured 108.7× throughput speedup exceeds the 35× projection made at M1. The M1 projection assumed a conservative AXI round-trip latency of ~7 µs per transaction. The actual measured average is ~2.6 µs, because the accelerator completes compute in ~1.1 µs (so polling wait is short) and the AXI crossbar latency at 13 MHz is lower than the worst-case estimate. The GFLOP/s ratio (11.4×) is much lower than the throughput ratio (108.7×) because the hardware kernel executes only 24 FLOPs per update compared to the software's 10,350 FLOPs: the hardware accelerates only the measurement-correction step, not the full predict+update cycle implemented in the Eigen software path.

Energy per update is estimated by multiplying the synthesis dynamic power estimate (0.423 W) by the per-update runtime. For the hardware accelerator: 0.423 W × 87.8 µs = 37.1 µJ. For the software baseline, the SweRV EL2 dynamic power at 13 MHz on the Artix-7 is estimated at ~0.2 W (CPU core plus memory, based on Nexys A7 datasheet typical figures at that activity level); 0.2 W × 9.551 ms = 1,910 µJ. The hardware accelerator is therefore approximately 51× more energy-efficient per Kalman update. This ratio is lower than the throughput speedup (108.7×) because the hardware instantaneous power (0.423 W) is more than double the estimated software path power (0.2 W).

The dominant bottleneck is AXI transaction latency. The accelerator compute time (72 cycles at 100 MHz = 720 ns internal, plus AXI setup ~1.1 µs total) is only 1.3% of the 87.8 µs per-update time. The remaining 98.7% is consumed by 34 AXI MMIO round-trips. The design sits 33.6× below the AXI bandwidth ceiling on the roofline chart (Figure 2). Adding state retention would reduce the transaction count from 34 to approximately 5 per update, which at the same ~2.6 µs/transaction average would reduce per-update latency to ~13 µs and yield a cumulative speedup of ~740× over the RVfpgaEL2 baseline.

9. What Did Not Work

Four significant setbacks occurred during the four-milestone project span. Each required a substantial design or scope change.

First, the M3 RTL failed timing badly. The Newton–Raphson S^{-1} computation chained two f64_mul and one f64_add in a single clock cycle—a 92-level logic path with a 60.38 ns data-path delay versus a 10 ns budget (WNS = −50.4 ns). The initial response was to lower the target clock to ~65 MHz, which would have closed timing without RTL changes but would have halved throughput and worsened the MMIO-latency bottleneck. Instead, pipeline registers were inserted: f64_mul was expanded from 2 to 3 stages (adding a mantissa pre-registration stage to break the LUT→DSP fanout path) and f64_add was expanded from 3 to 4 stages (adding a leading-zero count register stage to break the LZ-encoder→normalise fan-out path). The kalman_update FSM expanded from 11 to 51 states and gemm_systolic from 9 to 21 states to account for the increased pipeline latencies. This resolved timing: M4 WNS = +0.326 ns.

Second, the interface was changed twice. M1 selected PCIe Gen4 because the M1 profiling was done on an i7-11800H laptop where PCIe is the natural high-bandwidth interconnect. This was wrong for the actual target. The VeerWolf SoC on the Nexys A7 exposes only an AXI4-Lite peripheral bus; the SweRV EL2 issues MMIO accesses via load/store instructions mapped to AXI address space. There is no PCIe controller on the Nexys A7 board. The M2 interface.sv was accordingly rewritten as an AXI4-Lite slave. This also invalidated the M1 bandwidth calculation (PCIe Gen4 at 51.2 GB/s vs AXI4-Lite at 104 MB/s—a factor of ~490×). The correct lesson is that the interface choice must be derived from the target platform, not the profiling platform.

Third, the M2 fixed-point precision was insufficient. Q16.16 fixed-point was attempted to reduce DSP resource usage. The Kalman covariance matrix P converges toward small values over successive iterations, and the Newton–Raphson reciprocal of S requires sub-ULP accuracy over a dynamic range wider than Q16.16 can represent. P diverged on iteration 8 of 15. Switching to F64 resolved the divergence and produced correct results at ≤4 ULP on all 180 output checks, at the cost of 153 DSP48E1 tiles (63.75% of Artix-7 100T resources). No fixed-point quantisation analysis beyond this empirical test was performed; a more thorough analysis might have identified a wider format (e.g., Q32.32 or Q8.56) that could represent the required dynamic range while using fewer DSPs.

Fourth, state retention was not implemented. The post-M3 redesign analysis (Post-m3_Redesign.txt) identified state retention as the highest-ROI remaining improvement: retaining x and P internally between updates would reduce 34 MMIO transactions to approximately 5, yielding an estimated additional 6.8× speedup (to ~740× cumulative). This was not implemented in M4 due to scope constraints and the time required to redesign the axilite_slave register map, add internal state feedback paths in kalman_update, and update the driver and testbench accordingly. A DMA-based approach was also explored but ruled out: the VeerWolf SoC's DMA is inbound-only (host→peripheral) and cannot initiate reads of peripheral registers to implement the feedback loop.